THIẾT KẾ SƠ ĐỒ LỚP MIỀN NGHIỆP VỤ

(Domain Class Diagram – UML 2.5 Specification)

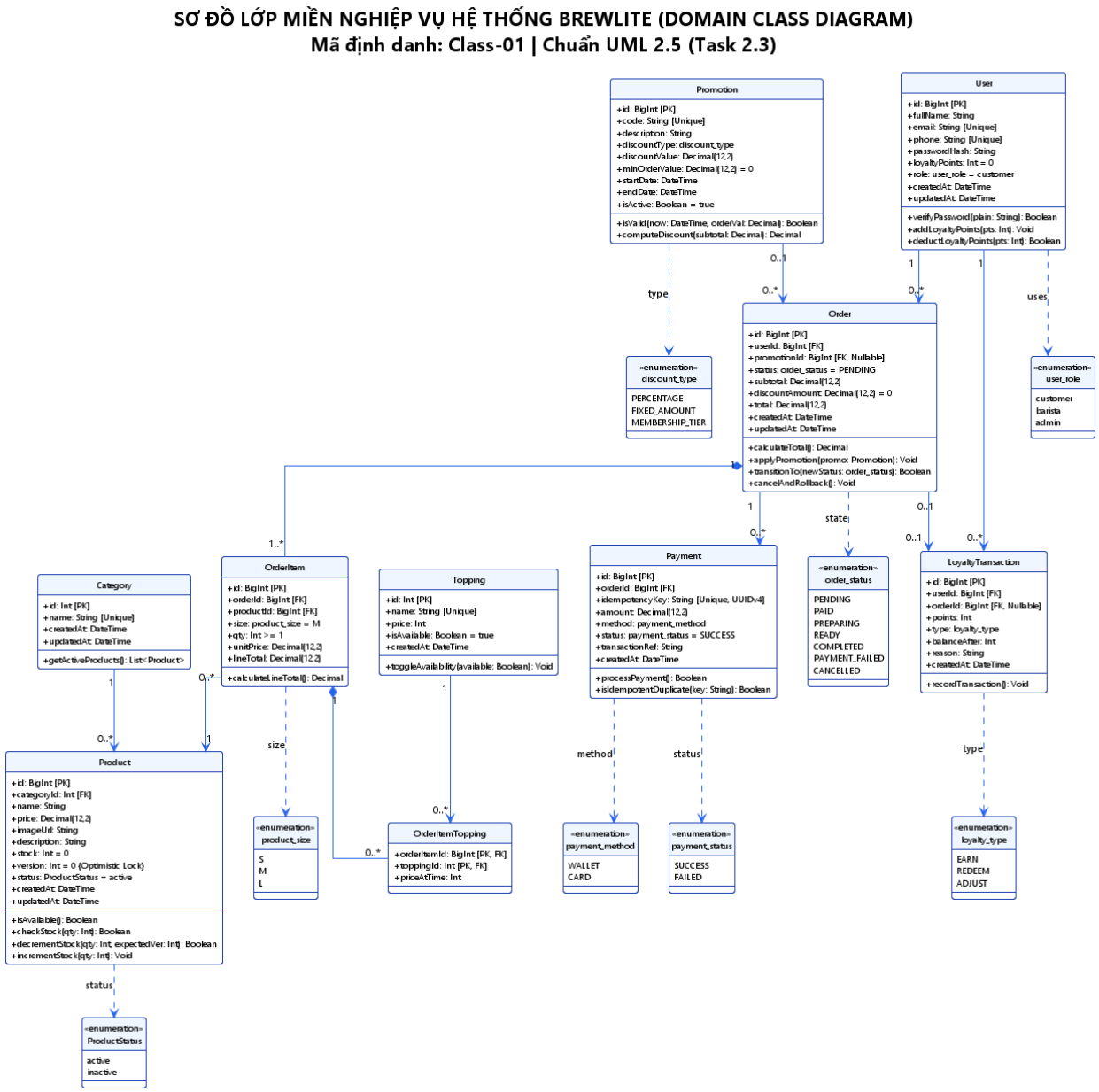
Dự án: Hệ thống Đặt đồ uống trực tuyến BrewLite
Học phần: Công nghệ Phần mềm (Software Engineering) – Assignment 2
Mã nhiệm vụ: Task 2.3 **Mã truy vết:** Class-01
Tệp nguồn PlantUML: `docs/diagrams/class-diagram.puml`
Tệp ảnh vector SVG: `docs/diagrams/class-diagram.svg`

1 Tổng quan mục tiêu & Tiêu chí nghiệm thu

Tài liệu này hiện thực hóa Task 2.3 của đồ án BrewLite, xây dựng sơ đồ lớp ở mức Domain Model (mô hình miền nghiệp vụ). Sơ đồ mô tả cấu trúc dữ liệu tĩnh, các thực thể kinh doanh, mối quan hệ cấu trúc, ràng buộc đa số (multiplicity), và các kiểu liệt kê trạng thái (enumerations).

2 Sơ đồ Lớp miền nghiệp vụ (Domain Class Diagram)

Sơ đồ được thiết kế bằng ngôn ngữ mô hình hóa PlantUML chuẩn UML 2.5:



Hình 1: Sơ đồ Lớp Miền Nghiệp Vụ Hệ Thống BrewLite (UML 2.5)

3 Đặc tả chi tiết các Lớp Miền Nghiệp Vụ (Domain Entities)

Hệ thống bao gồm **10 thực thể** (đáp ứng trọn vẹn 7 thực thể bắt buộc của đề bài cùng 3 thực thể hỗ trợ danh mục và topping theo mã nguồn thực tế):

Bảng 1: Danh mục 10 Lớp Miền Nghiệp Vụ trong Hệ Thống

STT	Tên Lớp	Thuộc tính chính & Kiểu dữ liệu	Phương thức nghiệp vụ chính

1	**User** *(Bắt buộc)*	`id: BigInt [PK]`
		`fullName: String`
		`email: String [Unique]`
		`phone: String [Unique]`
		`passwordHash: String`
		`loyaltyPoints: Int = 0`
		`role: user_role = customer`
		`createdAt, updatedAt: DateTime`
			`verifyPassword(plain): Boolean`
			`addLoyaltyPoints(pts): Void`
			`deductLoyaltyPoints(pts): Boolean`
			Quản lý tài khoản, mã hóa mật khẩu `bcrypt` và tích điểm thành viên.

2	**Product** *(Bắt buộc)*	`id: BigInt [PK]`
		`categoryId: Int [FK]`
		`name: String`
		`price: Decimal(12,2)`
		`imageUrl, description: String`
		`stock: Int = 0`
		`version: Int = 0 {Lock}`
		`status: ProductStatus = active`
		`createdAt, updatedAt: DateTime`
			`isAvailable(): Boolean`
			`checkStock(qty): Boolean`
			`decrementStock(qty, ver): Boolean`
			`incrementStock(qty): Void`
			Quản lý tồn kho đồ uống; kiểm soát tranh chấp bằng khóa lạc quan qua trường `version`.

3	**Order** *(Bắt buộc)*	`id: BigInt [PK]`
		`userId: BigInt [FK]`
		`promotionId: BigInt [FK, Null]`
		`status: order_status = PENDING`
		`subtotal, total: Decimal(12,2)`
		`discountAmount: Decimal(12,2)`
		`createdAt, updatedAt: DateTime`
			`calculateTotal(): Decimal`
			`applyPromotion(promo): Void`
			`transitionTo(newStatus): Boolean`
			`cancelAndRollback(): Void`
			Thực thi máy trạng thái 7 bước, tính tổng tiền và hoàn trả kho nếu hủy.

4	**OrderItem** *(Bắt buộc)*	`id: BigInt [PK]`
		`orderId: BigInt [FK]`
		`productId: BigInt [FK]`
		`size: product_size = M`
		`qty: Int >= 1`
		`unitPrice, lineTotal: Decimal(12,2)`
			`calculateLineTotal(): Decimal`
			Từng dòng món đặt trong đơn hàng kèm cấu hình dung tích kích cỡ.

5	**Payment** *(Bắt buộc)*	`id: BigInt [PK]`
		`orderId: BigInt [FK]`
		`idempotencyKey: String [Unique]`
		`amount: Decimal(12,2)`
		`method: payment_method`
		`status: payment_status`
		`transactionRef: String`
		`createdAt: DateTime`
			`processPayment(): Boolean`
			`isIdempotentDuplicate(key): Boolean`
			Xử lý trừ tiền trực tuyến mô phỏng, bảo đảm chống trừ tiền trùng bằng khóa Idempotency duy nhất.

STT	Tên Lớp	Thuộc tính chính & Kiểu dữ liệu	Phương thức nghiệp vụ (tiếp theo)

6	**Promotion** *(Bắt buộc)*	`id: BigInt [PK]`
		`code: String [Unique]`
		`description: String`
		`discountType: discount_type`
		`discountValue, minOrderValue`
		`startDate, endDate: DateTime`
		`isActive: Boolean = true`
			`isValid(now, orderVal): Boolean`
			`computeDiscount(subtotal): Decimal`
			Quản lý mã giảm giá và chiết khấu ưu đãi theo hạng thành viên (Bạc, Vàng, Kim Cương).

7	**LoyaltyTransaction** *(Bắt buộc)*	`id: BigInt [PK]`
		`userId: BigInt [FK]`
		`orderId: BigInt [FK, Nullable]`
		`points: Int`
		`type: loyalty_type`
		`balanceAfter: Int`
		`reason: String`
		`createdAt: DateTime`
			`recordTransaction(): Void`
			Lưu vết chi tiết lịch sử cộng/trừ điểm thưởng thành viên (tỷ lệ 10.000đ = 1 điểm).

8	**Category**	`id: Int [PK]`
		`name: String [Unique]`
		`createdAt, updatedAt: DateTime`
			`getActiveProducts(): List<Product>`
			Phân nhóm thực đơn (Cà phê, Trà sữa, Freeze).

9	**Topping**	`id: Int [PK]`
		`name: String [Unique]`
		`price: Int`
		`isAvailable: Boolean = true`
		`createdAt: DateTime`
			`toggleAvailability(status): Void`
			Nguyên liệu thêm kèm (trân châu, thạch, kem cheese).

10	**OrderItemTopping**	`orderItemId: BigInt [PK, FK]`
		`toppingId: Int [PK, FK]`
		`priceAtTime: Int`
			Liên kết trung gian lưu vết giá topping tại thời điểm khách đặt hàng.

4 Các kiểu Liệt kê Trạng thái & Nghiệp vụ (Enumerations)

Bảng 2: Danh mục 8 Kiểu Liệt Kê (Domain Enumerations)

STT	Tên Enum	Các giá trị hợp lệ	Ý nghĩa & Phạm vi áp dụng
1	`order_status`	`PENDING`, `PAID`, `PREPARING`, `READY`, `COMPLETED`, `PAYMENT_FAILED`, `CANCELLED`	Vòng đời 7 trạng thái của đơn hàng theo máy trạng thái FSM.
2	`payment_method`	`WALLET`, `CARD`	Phương thức thanh toán mô phỏng (Ví điện tử hoặc Thẻ).

STT	Tên Enum	Các giá trị hợp lệ	Ý nghĩa (tiếp theo)
3	`payment_status`	`SUCCESS`, `FAILED`	Kết quả xử lý trừ tiền từ cổng thanh toán mô phỏng.
4	`product_size`	`S`, `M`, `L`	Kích cỡ đồ uống (Size S cơ sở, M +5.000đ, L +10.000đ).
5	`ProductStatus`	`active`, `inactive`	Trạng thái kinh doanh bật/tắt của món đồ uống.
6	`user_role`	`customer`, `barista`, `admin`	Phân quyền truy cập người dùng trong hệ thống (RBAC).
7	`discount_type`	`PERCENTAGE`, `FIXED_AMOUNT`, `MEMBERSHIP_TIER`	Phân loại hình thức chiết khấu khuyến mãi.
8	`loyalty_type`	`EARN`, `REDEEM`, `ADJUST`	Bản chất giao dịch điểm thưởng (Tích điểm khi đơn PAID, Đổi điểm thưởng).

5 Quan hệ & Ràng buộc Đa số (Multiplicity)

Các mối quan hệ cấu trúc tuân thủ nghiêm ngặt ngữ nghĩa UML 2.5:

- `Category` $1 - 0..*$ `Product` **(Association):** Một danh mục chứa từ 0 đến nhiều sản phẩm. Một sản phẩm thuộc về một danh mục duy nhất.
- `User` $1 - 0..*$ `Order` **(Association):** Một khách hàng có thể khởi tạo 0 hoặc nhiều đơn hàng.
- `Order` 1 $\blacklozenge$– $1..*$ `OrderItem` **(Composition – Hợp thành sống còn):** Mỗi đơn hàng bắt buộc phải có từ 1 đến nhiều món. Khi đơn hàng bị hủy bỏ triệt để khỏi hệ thống, các `OrderItem` con tự động bị xóa theo (`onDelete: Cascade`).
- `OrderItem` 1 $\blacklozenge$– $0..*$ `OrderItemTopping` **(Composition):** Mỗi món đồ uống có thể chọn từ 0 đến nhiều loại topping đi kèm.
- `Order` $1 - 0..*$ `Payment` **(Association):** Một đơn hàng liên kết với các lượt thanh toán. Cho phép lưu vết các lượt thanh toán thất bại (`PAYMENT_FAILED`) trước khi có 1 lần thanh toán thành công (`PAID`).
- `Promotion` $0..1 - 0..*$ `Order` **(Association):** Một đơn hàng có thể không áp dụng hoặc áp dụng tối đa một mã ưu đãi/chiết khấu thành viên.
- `User` $1 - 0..*$ `LoyaltyTransaction` **(Association):** Một tài khoản khách hàng sở hữu toàn bộ lịch sử các giao dịch điểm tích lũy.
- `Order` $0..1 - 0..1$ `LoyaltyTransaction` **(Association):** Mỗi đơn hàng thanh toán thành công (`PAID`) phát sinh duy nhất một giao dịch cộng điểm thưởng (10.000đ = 1 điểm).

6 Chứng minh Tiêu chí Nghiệm thu Khắt khe (Zero Contradiction)

Bảng 3: Bảng Đối Chiếu Tính Nhất Quán Tuyệt Đối Của Sơ Đồ Lớp

STT	Tài liệu đối chiếu	Điểm ràng buộc kỹ thuật	Minh chứng tính nhất quán trong Class Diagram
1	**Cơ sở dữ liệu** `schema.prisma`	Khóa chính, khóa ngoại, kiểu dữ liệu, trường khóa lạc quan, trường chống trùng thanh toán.	Trùng khớp 100% các kiểu: `BigInt`, `Decimal(12,2)`, `DateTime`. Trường `Product.version` thực thi Optimistic Concurrency Control. Trường `Payment.idempotencyKey` gán thuộc tính `[Unique]`.
2	**Đặc tả yêu cầu (SRS)** `srs.md`	Các mã chức năng: FR-01, FR-02, FR-03, FR-05, FR-06, FR-07, FR-08, FR-10, FR-11.	FR-01/02 qua `Product`, `Category`, `Topping`. FR-03 qua `OrderItem`. FR-05 qua `Order.status = PENDING`. FR-06 qua `Payment.idempotencyKey`. FR-07/11 qua `LoyaltyTransaction` và `Promotion`. FR-08 qua `order_status` FSM.
3	**Sơ đồ tuần tự** `sequence-checkout.puml`	Thứ tự gọi hàm, transaction tạo đơn và xác thực thanh toán qua cổng mô phỏng.	Khớp chính xác các tham số gọi trong Sequence: hàm tạo đơn `POST /orders` tạo đối tượng `Order` & `OrderItem`; hàm `POST /payments` kiểm tra trùng `idempotencyKey` và nạp giao dịch `LoyaltyTransaction`.
4	**Sơ đồ hoạt động** `activity-order-flow.puml`	Nhánh kiểm tra tồn kho, nhánh hủy đơn và rollback số lượng tồn kho khi thanh toán lỗi.	Nhánh kiểm tra tồn kho khớp với `Product.checkStock()`. Nhánh hủy đơn quá hạn/thanh toán thất bại khớp với phương thức `Order.cancelAndRollback()` kích hoạt `Product.incrementStock()`.